

# Chapter 6: Second-Order Methods

Algorithms for Optimization, 2<sup>nd</sup> ed. (2026)

Kochenderfer & Wheeler

Lecture Slides

# Outline

- 1 Introduction
- 2 Newton's Method
- 3 Secant Method
- 4 Levenberg-Marquardt Algorithm
- 5 Levenberg-Marquardt for Sum of Squares
- 6 Quasi-Newton Methods
- 7 Summary & Exercises

# Motivation: Why Second-Order Information? I

When we minimise a function  $f(\mathbf{x})$ , we need to decide *two* things at every iteration: in which *direction* to move, and by *how far* to move. First-order methods answer only the first question by consulting the gradient  $\nabla f(\mathbf{x})$  (nabla  $f$ , i.e. the vector of partial derivatives). They tell us the direction of steepest ascent, so we step in the opposite direction. However, the gradient gives no information about the curvature of  $f$  – it cannot say whether the function is a gentle slope or a sharp valley. As a result, first-order methods often take many small, cautious steps in highly curved regions.

# Motivation: Why Second-Order Information? II

## First-Order Methods: Direction Without Curvature

A first-order method uses  $\nabla f(\mathbf{x})$  (nabla  $f$ , the gradient vector) to determine the search direction. The update takes the form

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \nabla f(\mathbf{x}^{(k)}),$$

where  $\mathbf{x}^{(k)}$  ( $\mathbf{x}$  superscript  $k$ ) is the current point at iteration  $k$ ,  $\alpha^{(k)}$  (alpha superscript  $k$ ) is the step size chosen at iteration  $k$ , and  $\nabla f(\mathbf{x}^{(k)})$  is the gradient evaluated there. The gradient tells us the direction of steepest ascent, so taking a step opposite to it decreases  $f$ . However, the gradient alone cannot tell us the *optimal* step size: a flat function and a sharply curved function have the same gradient at an intermediate point, yet require very different step sizes.

# Motivation: Why Second-Order Information? III

## Second-Order Methods: Building a Local Quadratic Model

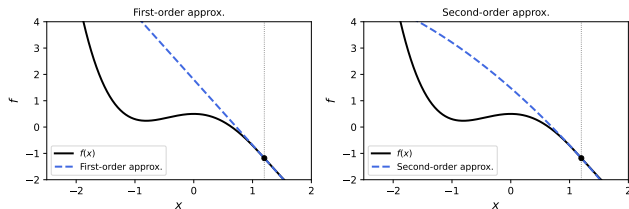
Second-order methods augment the gradient with the **Hessian**  $\mathbf{H}(\mathbf{x}) = \nabla^2 f(\mathbf{x})$  (nabla-squared  $f$ ), which is the matrix of second partial derivatives. The entry  $H_{ij}$  ( $H$  sub  $ij$ ) is  $\partial^2 f / \partial x_i \partial x_j$ , encoding how the  $i$ -th component of the gradient changes as we move along the  $j$ -th coordinate. With both gradient and Hessian in hand we can fit a *quadratic* (bowl-shaped or saddle-shaped) local model to  $f$ , and a bowl has a unique lowest point that we can find analytically – no trial-and-error step size needed.

## Key Insight

A quadratic function has exactly one stationary point (where derivative is zero), and we can jump there in a single step. If  $f$  is well approximated by a quadratic near the current iterate, second-order methods converge much faster than gradient-only methods.

# Motivation: Why Second-Order Information? IV

First- vs. Second-Order Approximations



Top: a linear (first-order) approximation misses the minimum – the tangent line at the current point points away from where the function actually bottoms out. Bottom: a quadratic (second-order) approximation curves down to enclose the minimum, allowing us to step directly there.

## What the Hessian $\mathbf{H}$ Tells Us

- If  $\mathbf{H}$  is *positive definite* (all eigenvalues positive), the quadratic model is bowl-shaped and has a unique minimum. The Newton step jumps directly to it.
- If  $\mathbf{H}$  is *indefinite* (mixed-sign eigenvalues), the model is saddle-shaped and the Newton step may go uphill. Damping (as in Levenberg-Marquardt) repairs this.
- The diagonal entries  $H_{ii}$  measure curvature along each coordinate axis: a large  $H_{ii}$  means the function is sharply curved along  $x_i$ , so a small step is warranted in that direction.

# Quadratic Approximation via Taylor Expansion I

The mathematical foundation of all second-order methods is the **Taylor expansion**: any smooth function can be approximated near a given point by a polynomial. Stopping at the second-order term gives a quadratic approximation  $q$  that is much more accurate than the linear (gradient-only) approximation, yet still has a simple analytical minimum.

# Quadratic Approximation via Taylor Expansion II

## Univariate: Second-Order Taylor Expansion

For a function  $f : \mathbb{R} \rightarrow \mathbb{R}$  of a single variable, the second-order Taylor expansion around the current iterate  $x^{(k)}$  ( $x$  at iteration  $k$ ) is:

$$q(x) = f(x^{(k)}) + (x - x^{(k)}) f'(x^{(k)}) + \frac{(x - x^{(k)})^2}{2} f''(x^{(k)})$$

### Symbol guide:

- $x^{(k)}$  is the expansion point – the current best guess.
- $f'(x^{(k)})$  ( $f$  prime at  $x^{(k)}$ ) is the first derivative – the slope of  $f$  at the expansion point.
- $f''(x^{(k)})$  ( $f$  double-prime at  $x^{(k)}$ ) is the second derivative – the curvature, i.e. how fast the slope itself is changing.
- $(x - x^{(k)})$  is the displacement from the expansion point to the query point  $x$ .



# Quadratic Approximation via Taylor Expansion III

## How to Read This Formula

Think of  $q(x)$  as a parabola centred at  $x^{(k)}$ . The first-derivative term tilts the parabola left or right (the linear trend); the second-derivative term determines whether the parabola opens upward (positive curvature,  $f'' > 0$ ) or downward (negative curvature,  $f'' < 0$ ). If  $f'' > 0$  the parabola has a unique minimum we can reach by setting  $q'(x) = 0$ , which gives the Newton step.

# Quadratic Approximation via Taylor Expansion IV

## Multivariate: Second-Order Taylor Expansion

For  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  (a function of  $n$  ( $n$ ) variables), the expansion generalises to:

$$q(\mathbf{x}) = f(\mathbf{x}^{(k)}) + (\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{g}^{(k)} + \frac{1}{2}(\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{H}^{(k)}(\mathbf{x} - \mathbf{x}^{(k)})$$

### Symbol guide:

- $\mathbf{x} \in \mathbb{R}^n$  (bold  $\mathbf{x}$ ) is the  $n$ -dimensional input vector;  $\mathbf{x}^{(k)}$  is its value at iteration  $k$ .
- $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)}) \in \mathbb{R}^n$  (bold  $\mathbf{g}$ , the gradient vector) contains the  $n$  partial derivatives of  $f$  at  $\mathbf{x}^{(k)}$ .
- $\mathbf{H}^{(k)} = \nabla^2 f(\mathbf{x}^{(k)}) \in \mathbb{R}^{n \times n}$  (bold  $\mathbf{H}$ , the Hessian matrix) is the  $n \times n$  matrix of second partial derivatives; entry  $(i, j)$  is  $\partial^2 f / \partial x_i \partial x_j$ .
- The superscript  $\top$  denotes the transpose, turning a column vector into a row vector so the products are dimensionally consistent.
- The term  $(\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{H}^{(k)}(\mathbf{x} - \mathbf{x}^{(k)})$  is a *quadratic form* – a scalar that encodes the curvature of  $f$  in the direction of displacement  $(\mathbf{x} - \mathbf{x}^{(k)})$ .

## Quadratic Approximation via Taylor Expansion V

Setting the gradient of  $q$  with respect to  $\mathbf{x}$  to zero (i.e.  $\nabla q(\mathbf{x}) = \mathbf{0}$ ) and solving for  $\mathbf{x}$  gives the **Newton step** directly. This is the central computation shared by all second-order methods in this chapter.

## 6.1 Newton's Method: The Core Idea I

Newton's method asks: *if the function looks like a quadratic bowl right now, where is the bottom of that bowl?* It then steps directly there. Because real functions are not perfectly quadratic, we repeat: rebuild the quadratic approximation at the new point, find its bottom, step there, and so on. Near a true minimum the approximation becomes increasingly accurate, which is why convergence accelerates dramatically as we approach the solution.

## 6.1 Newton's Method: The Core Idea II

### Univariate Newton Step

Setting  $q'(x) = 0$  in the univariate Taylor expansion and solving for the minimiser  $x^{(k+1)}$  yields the update rule:

$$x^{(k+1)} \leftarrow x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$

Reading this formula: we subtract from the current point  $x^{(k)}$  the ratio of the slope  $f'$  to the curvature  $f''$ . If the slope is large and the curvature is also large (sharp bowl), the step is moderate. If the curvature is small (shallow, nearly flat), the denominator is small and the step is large – reflecting the fact that a shallow bowl's minimum is far away. The condition  $f''(x^{(k)}) > 0$  (positive curvature) is required for this to be a *descent* step; if  $f''(x^{(k)}) < 0$  the parabola opens downward and the step goes uphill.

## 6.1 Newton's Method: The Core Idea III

### Multivariate Newton Step

In  $n$  dimensions, setting  $\nabla q(\mathbf{x}) = \mathbf{0}$  and solving for  $\mathbf{x}^{(k+1)}$  gives:

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \left(\mathbf{H}^{(k)}\right)^{-1} \mathbf{g}^{(k)}$$

Here  $(\mathbf{H}^{(k)})^{-1}$  ( $\mathbf{H}$  inverse) is the inverse of the Hessian matrix, and  $\mathbf{g}^{(k)}$  (bold  $g$ ) is the gradient vector at  $\mathbf{x}^{(k)}$ . The product  $(\mathbf{H}^{(k)})^{-1} \mathbf{g}^{(k)}$  is the *Newton direction*: it is the gradient pre-multiplied by the inverse curvature, which rescales each gradient component by the appropriate curvature in that direction. In practice we never explicitly invert  $\mathbf{H}^{(k)}$  (matrix inversion is numerically unstable and slower than necessary). Instead we solve the linear system

$$\mathbf{H}^{(k)} \mathbf{d} = -\mathbf{g}^{(k)}$$

for the Newton direction  $\mathbf{d}$  (bold  $d$ ), then update:  $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}$ .

## 6.1 Newton's Method: The Core Idea IV

### When Is the Newton Step a Descent Direction?

The Newton direction  $\mathbf{d} = -(\mathbf{H}^{(k)})^{-1}\mathbf{g}^{(k)}$  points downhill (i.e. it is a descent direction) if and only if  $\mathbf{H}^{(k)}$  is *positive definite* – meaning every eigenvalue (the principal curvatures) is strictly positive. When this holds,  $f$  is bowl-shaped locally and the Newton step guarantees a decrease. When  $\mathbf{H}^{(k)}$  is indefinite or negative definite, the Newton step may increase  $f$ , and we need to augment the method (e.g. with a line search or damping).

## 6.1 Newton's Method: The Core Idea V

### Algorithm: Newton's Method (Step by Step)

- ➊ **Initialise.** Choose a starting point  $\mathbf{x}^{(1)}$  and a convergence tolerance  $\epsilon$  (epsilon, a small positive number such as  $10^{-8}$ ) representing the maximum acceptable gradient magnitude.
- ➋ **Compute gradient.** Evaluate  $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$ .
- ➌ **Check convergence.** If  $\|\mathbf{g}^{(k)}\| \leq \epsilon$  (the Euclidean norm of the gradient is small enough), stop and return  $\mathbf{x}^{(k)}$ .
- ➍ **Compute Hessian.** Evaluate  $\mathbf{H}^{(k)} = \nabla^2 f(\mathbf{x}^{(k)})$ .
- ➎ **Solve for Newton direction.** Find  $\mathbf{d}$  by solving  $\mathbf{H}^{(k)}\mathbf{d} = -\mathbf{g}^{(k)}$  (a linear system, not a matrix inversion).
- ➏ **Update.** Set  $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}$  and go to step 2.



## 6.1 Newton's Method: The Core Idea VI

### Convergence Properties

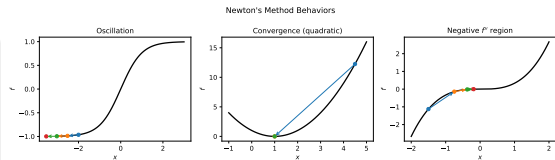
Newton's method exhibits **quadratic convergence** in a neighbourhood of a local minimum: if the error at iteration  $k$  is  $e_k = \|\mathbf{x}^{(k)} - \mathbf{x}^*\|$ , then  $e_{k+1} \leq C \cdot e_k^2$  for some constant  $C > 0$ . Concretely, if you are within 0.1 of the solution, the next step brings you within roughly 0.01, the step after that within 0.0001, and so on – the number of accurate digits roughly doubles each iteration. On a strictly quadratic function (where the second-order Taylor model is exact) Newton's method converges in **exactly one step** regardless of the starting point. The computational cost per iteration is  $O(n^3)$  (big-O n-cubed) for solving the  $n \times n$  linear system, which can be expensive for large  $n$  but is often acceptable when  $n$  is small to moderate.

## 6.1 Newton's Method: The Core Idea VII

### Potential Problems and Failure Modes

Newton's method can fail or behave badly in three ways:

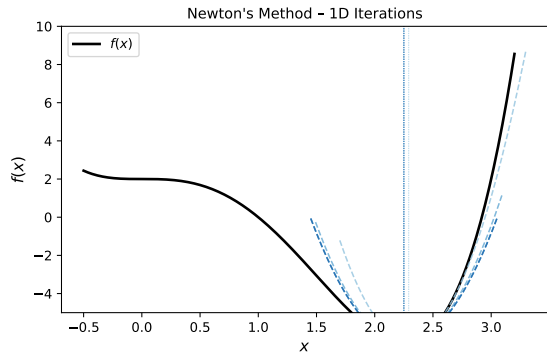
- 1 **Zero or vanishing second derivative.** If  $f''(x^{(k)}) \approx 0$  (the function is nearly flat), the denominator of the Newton step is near zero, and the step size becomes enormous. The iterate may jump far outside the region where the quadratic approximation is valid.
- 2 **Negative curvature.** If  $f''(x^{(k)}) < 0$  (or  $\mathbf{H}^{(k)}$  is indefinite), the parabola opens downward. The Newton step moves toward a *maximum* of the quadratic model, not a minimum. The iterate may oscillate or diverge.
- 3 **Starting far from the minimum.** Even with positive curvature, if the quadratic model is a poor approximation (the iterate is far from the



Three behaviours of Newton's method in 1D. Left: oscillation (iterates bounce back and forth across the minimum because the curvature is not well captured). Centre: clean convergence (iterates march steadily toward the minimum). Right: divergence near a region of negative curvature, where the Newton step points uphill.

# Newton's Method: Iteration Visualised I

The figure below shows Newton's method converging on a 1D function. At each iteration, we (1) build the quadratic approximation  $q(x)$  centred at the current point (dashed parabola), then (2) jump to the minimum of  $q$ , which becomes the next iterate. Close to the true minimum the parabola fits the function almost perfectly, so convergence accelerates dramatically.



Each dashed curve is the local quadratic approximation  $q(x)$  at the current iterate. The solid dot marks where  $q'(x) = 0$ , which

## What to Look for in This Plot

Observe how the spacing between successive iterates shrinks rapidly: early steps are large (the quadratic approximation is coarse), while later steps are tiny (the approximation nearly matches the true function). This geometric shrinkage – where each step's error is roughly the *square* of the previous error – is the hallmark of quadratic convergence.

## Termination Criteria

We stop when any of the following is satisfied:

- 1  $\|\mathbf{g}^{(k)}\| \leq \epsilon$  (gradient magnitude below

# Newton's Method: Python Implementation

Listing 1: Newton's method (multivariate) in Python

```
1 import numpy as np
2 from numpy.linalg import solve, norm
3
4 def newtons_method(f, grad_f, hess_f, x0, eps=1e-8, max_iter=100):
5     """
6     Newton's method for unconstrained minimization.
7     f          : objective function  $\mathbb{R}^n \rightarrow \mathbb{R}$ 
8     grad_f     : gradient function  $\mathbb{R}^n \rightarrow \mathbb{R}^n$ 
9     hess_f     : Hessian function  $\mathbb{R}^n \rightarrow \mathbb{R}^{\{n \times n\}}$ 
10    x0         : initial point (numpy array)
11    """
12    x = np.array(x0, dtype=float)
13    history = [x.copy()]
14    for k in range(max_iter):
15        g = grad_f(x)
16        if norm(g) < eps:
17            break
18        H = hess_f(x)
19        # Solve  $H*d = -g$  (more stable than inverting H)
20        d = solve(H, -g)
21        x = x + d
22        history.append(x.copy())
23    return x, history
24
25 # --- Example: Booth's function  $f(x) = (x_1+2x_2-7)^2 + (2x_1+x_2-5)^2$  ---
```

## Worked Example: Newton's Method on Booth's Function I

Booth's function is a classic 2D test problem that is quadratic in both variables. Because it is exactly quadratic, Newton's method – which builds a quadratic model – captures  $f$  perfectly and converges in a *single step* from any starting point. Working through the algebra manually is a good way to build intuition for what the Newton update is actually computing.

### Booth's Function

$$f(\mathbf{x}) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$$

Here  $x_1$  (x sub 1) and  $x_2$  (x sub 2) are the two components of the input vector  $\mathbf{x} = [x_1, x_2]^T$ . The function is the sum of two squared linear expressions; geometrically it is an elliptic paraboloid (a bowl tilted in the  $x_1$ - $x_2$  plane). The unique global minimum is at  $\mathbf{x}^* = [1, 3]^T$ , where both squared terms equal zero.

## Worked Example: Newton's Method on Booth's Function II

### Gradient of Booth's Function

Differentiating with respect to  $x_1$  and  $x_2$  respectively:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2(x_1 + 2x_2 - 7) \cdot 1 + 2(2x_1 + x_2 - 5) \cdot 2 \\ 2(x_1 + 2x_2 - 7) \cdot 2 + 2(2x_1 + x_2 - 5) \cdot 1 \end{bmatrix} = \begin{bmatrix} 10x_1 + 8x_2 - 34 \\ 8x_1 + 10x_2 - 38 \end{bmatrix}$$

## Worked Example: Newton's Method on Booth's Function III

### Hessian of Booth's Function (Constant!)

Differentiating the gradient again:

$$\mathbf{H}(\mathbf{x}) = \begin{bmatrix} \partial^2 f / \partial x_1^2 & \partial^2 f / \partial x_1 \partial x_2 \\ \partial^2 f / \partial x_2 \partial x_1 & \partial^2 f / \partial x_2^2 \end{bmatrix} = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$$

Notice that the Hessian is *constant* – it does not depend on  $\mathbf{x}$  at all. This is the defining property of a quadratic function: its second derivatives are the same everywhere. Both eigenvalues of this matrix are positive (18 and 2), confirming the function is strictly convex.

## Worked Example: Newton's Method on Booth's Function IV

### One Newton Step from $\mathbf{x}^{(1)} = [9, 8]^\top$

We start at  $\mathbf{x}^{(1)} = [9, 8]^\top$  (a point far from the minimum).

**Step 1: compute the gradient at  $\mathbf{x}^{(1)}$ .**

$$\mathbf{g}^{(1)} = \begin{bmatrix} 10(9) + 8(8) - 34 \\ 8(9) + 10(8) - 38 \end{bmatrix} = \begin{bmatrix} 90 + 64 - 34 \\ 72 + 80 - 38 \end{bmatrix} = \begin{bmatrix} 120 \\ 114 \end{bmatrix}$$

**Step 2: solve  $\mathbf{H} \mathbf{d} = -\mathbf{g}^{(1)}$ .**

$$\begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix} \mathbf{d} = -\begin{bmatrix} 120 \\ 114 \end{bmatrix}$$

The inverse of the constant  $2 \times 2$  Hessian is  $\mathbf{H}^{-1} = \frac{1}{36} \begin{bmatrix} 10 & -8 \\ -8 & 10 \end{bmatrix}$  (using the formula for  $2 \times 2$  matrix inverse with determinant  $10 \cdot 10 - 8 \cdot 8 = 36$ ). Therefore:

$$\mathbf{d} = -\mathbf{H}^{-1} \mathbf{g}^{(1)} = -\frac{1}{36} \begin{bmatrix} 10 & -8 \\ -8 & 10 \end{bmatrix} \begin{bmatrix} 120 \\ 114 \end{bmatrix} = -\frac{1}{36} \begin{bmatrix} 288 \\ 114 \cdot 10 - 120 \cdot 8 \end{bmatrix}$$

Working out the arithmetic:  $10 \cdot 120 - 8 \cdot 114 = 1200 - 912 = 288$  and



## 6.2 The Secant Method I

Newton's method requires the second derivative  $f''(x)$ , which must be computed analytically or numerically. For some functions this is expensive, error-prone, or simply inconvenient. The **secant method** sidesteps this by *approximating*  $f''$  from two recent evaluations of the first derivative  $f'$ , in the same way that a secant line through two points approximates the derivative.

### Second Derivative Approximation

Given two iterates  $x^{(k-1)}$  and  $x^{(k)}$  ( $x$  at the previous iteration and the current iteration), and the corresponding first-derivative values  $f'(x^{(k-1)})$  and  $f'(x^{(k)})$ , we approximate the second derivative as:

$$f''(x^{(k)}) \approx \frac{f'(x^{(k)}) - f'(x^{(k-1)})}{x^{(k)} - x^{(k-1)}}$$

This is a **finite difference** approximation: it measures how much the slope  $f'$  has changed between the two points, divided by how far apart those points are. When  $x^{(k)}$  and  $x^{(k-1)}$  are close, this ratio converges to the true second derivative.

## 6.2 The Secant Method II

### Secant Update Rule

Substituting this approximation into the Newton update  $x^{(k+1)} = x^{(k)} - f'(x^{(k)})/f''(x^{(k)})$  gives the secant step:

$$x^{(k+1)} \leftarrow x^{(k)} - \frac{x^{(k)} - x^{(k-1)}}{f'(x^{(k)}) - f'(x^{(k-1)})} f'(x^{(k)})$$

#### Symbol guide:

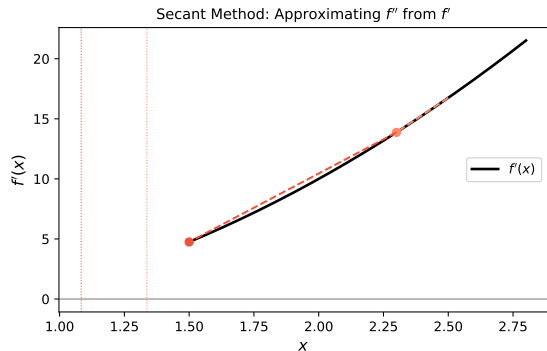
- $x^{(k)}$  and  $x^{(k-1)}$ : the two most recent iterates.
- $f'(x^{(k)})$  and  $f'(x^{(k-1)})$ : the first derivative evaluated at those two iterates.
- The ratio  $(x^{(k)} - x^{(k-1)})/(f'(x^{(k)}) - f'(x^{(k-1)}))$ : the reciprocal of the approximate second derivative, i.e. the approximate inverse curvature.

## 6.2 The Secant Method III

### How to Read This Formula

The secant step is exactly Newton's step, but with the true second derivative replaced by an approximation built from two successive gradient evaluations. The approximation improves as the iterates approach each other (and both approach the minimum), which is why the method still achieves better-than-linear convergence – it converges at order  $\approx 1.618$  (the golden ratio), compared to order 1 (linear) for gradient descent and order 2 (quadratic) for Newton's method.

## 6.2 The Secant Method IV



The secant line (dashed) passes through the two most recent evaluations of  $f'$ . Its slope approximates  $f''$ ; where it crosses zero gives the next iterate. As the iterates converge, the secant line becomes an increasingly good proxy for the tangent to  $f'$ .

### Secant Algorithm

- 1 Choose an initial point  $x^{(1)}$  and a small step size  $\alpha$  (alpha) to generate the second starting point:  $x_{\text{prev}} = x^{(1)} + \alpha f'(x^{(1)})$ .
- 2 Evaluate  $g_{\text{prev}} = f'(x_{\text{prev}})$  and set  $x = x^{(1)}$ .
- 3 **Repeat until convergence:**
  - 1 Compute  $g = f'(x)$ .
  - 2 If  $|g| \leq \epsilon$  (epsilon), stop.
  - 3 Compute the secant step:
$$x' = x - \frac{x - x_{\text{prev}}}{g - g_{\text{prev}}} \cdot g.$$
  - 4 Shift:  $x_{\text{prev}} \leftarrow x$ ,  $g_{\text{prev}} \leftarrow g$ .
  - 5 Update:  $x \leftarrow x'$ .

### Properties and Trade-offs

The secant method requires only *first* derivatives (no  $f''$ ), but it needs one extra initialisation step to

# Secant Method: Python Implementation

Listing 2: Secant method for univariate minimization in Python

```
1 import numpy as np
2
3 def secant_method(fp, x0, alpha=1e-3, eps=1e-8, max_iter=200):
4     """
5     Secant method for univariate minimization.
6     fp : first derivative f'(x)
7     x0 : initial point
8     alpha: step for initializing previous point
9     """
10    x_prev = x0 + alpha * fp(x0) # small step in gradient direction
11    g_prev = fp(x_prev)
12    x = x0
13    history = [x]
14
15    for _ in range(max_iter):
16        g = fp(x)
17        if abs(g) < eps:
18            break
19        denom = g - g_prev
20        if abs(denom) < 1e-14:
21            break # avoid division by zero
22        x_new = x - (x - x_prev) / denom * g
23        x_prev, g_prev = x, g
24        x = x_new
25        history.append(x)
```

## 6.3 Levenberg-Marquardt Algorithm I

Newton's method is powerful but brittle: it fails when the Hessian is not positive definite, and it can diverge when the quadratic model is a poor approximation (far from the minimum). The **Levenberg-Marquardt (LM)** algorithm repairs both problems by adding a positive constant  $\delta$  (delta) to the Hessian's diagonal before solving. This “damping” pushes the effective curvature to be positive, making the modified Hessian positive definite and the resulting step a genuine descent direction. Moreover, by adjusting  $\delta$  at each iteration based on whether the last step was accepted, the algorithm smoothly interpolates between gradient descent (when far from the minimum, large  $\delta$ ) and Newton's method (when close to the minimum, small  $\delta$ ).

## 6.3 Levenberg-Marquardt Algorithm II

### LM Update: Plain Damping

The simplest form adds  $\delta$  (delta, the damping factor) times the identity matrix  $\mathbf{I}$  ( $\mathbf{I}$ , a matrix with ones on the diagonal and zeros elsewhere) to the Hessian:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \left( \mathbf{H}^{(k)} + \delta \mathbf{I} \right)^{-1} \mathbf{g}^{(k)}$$

#### Symbol guide:

- $\mathbf{H}^{(k)}$  (bold H): the Hessian matrix at the current iterate.
- $\delta$  (delta): the damping factor, a positive scalar we adjust between iterations.
- $\mathbf{I}$  (bold I): the  $n \times n$  identity matrix – the matrix equivalent of the number 1.
- Adding  $\delta \mathbf{I}$  to  $\mathbf{H}^{(k)}$  shifts every eigenvalue of  $\mathbf{H}$  up by  $\delta$ ; with large enough  $\delta$  all eigenvalues become positive, guaranteeing positive definiteness.

## 6.3 Levenberg-Marquardt Algorithm III

### How to Read the Damping Formula

When  $\delta \rightarrow 0$  (delta approaches zero), the damped matrix approaches  $\mathbf{H}^{(k)}$  and the step approaches the pure Newton step. When  $\delta \rightarrow \infty$  (delta grows large), the damped matrix is dominated by  $\delta \mathbf{I}$  and the step  $-(\delta \mathbf{I})^{-1} \mathbf{g} \approx -(1/\delta) \mathbf{g}$  approaches a tiny gradient descent step. By adaptively tuning  $\delta$ , LM gets the rapid convergence of Newton near the minimum while behaving like the safer gradient descent when far away.

### LM Update: Diagonal Scaling (Improved)

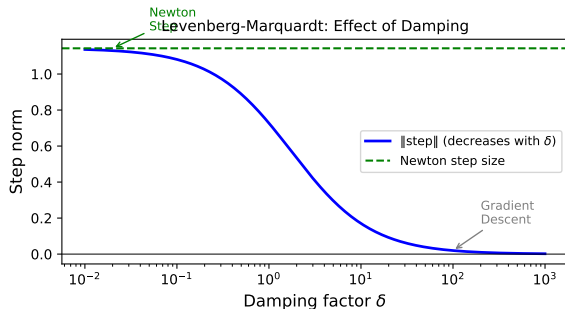
Rather than adding a uniform  $\delta$  to all diagonal entries, we can scale the damping by the existing curvature on each axis:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \left( \mathbf{H}^{(k)} + \delta \operatorname{diag}(\mathbf{H}^{(k)}) \right)^{-1} \mathbf{g}^{(k)}$$

Here  $\operatorname{diag}(\mathbf{H}^{(k)})$  is the diagonal matrix formed from the diagonal entries of  $\mathbf{H}^{(k)}$ . Dimensions with large curvature (large  $H_{ii}$ ) get more damping; dimensions with small curvature get less. This produces *scale-invariant* damping that works well even when the variables have very different magnitudes.



## 6.3 Levenberg-Marquardt Algorithm IV



As the damping factor  $\delta$  (delta) increases from left to right, the step size shrinks from the pure Newton size toward zero. At very large  $\delta$  the method behaves identically to a scaled gradient descent step.

### Adaptive Damping Strategy

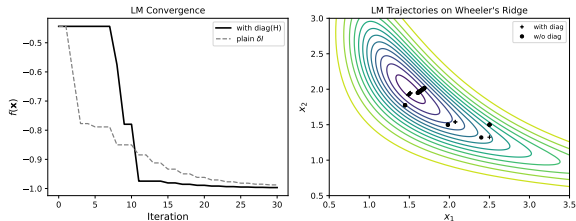
- 1 At each iteration, compute the candidate step  $\mathbf{x}'$  using the current  $\delta$ .
- 2 If  $f(\mathbf{x}') < f(\mathbf{x})$  (**step accepted**): the quadratic model was a good approximation, so reduce  $\delta$  (move toward Newton):  
 $\delta \leftarrow \delta \cdot \gamma_{\text{acc}}$  where  $\gamma_{\text{acc}} \in (0, 1)$  (typically 0.1).
- 3 If  $f(\mathbf{x}') \geq f(\mathbf{x})$  (**step rejected**): the approximation was poor, so increase  $\delta$  (move toward gradient descent):  $\delta \leftarrow \delta \cdot \gamma_{\text{rej}}$  where  $\gamma_{\text{rej}} > 1$  (typically 10).

Here  $\gamma_{\text{acc}}$  (gamma subscript “acc”) and  $\gamma_{\text{rej}}$  (gamma subscript “rej”) are hyperparameters (user-set constants) governing how aggressively  $\delta$  changes.

## 6.3 Levenberg-Marquardt Algorithm V

### Parameter Guidelines

| Parameter                         | Meaning                                                 | Typical value |
|-----------------------------------|---------------------------------------------------------|---------------|
| $\delta$                          | initial damping factor (starts moderate)                | 1.0           |
| $\gamma_{\text{acc}}$ (gamma-acc) | factor to <i>reduce</i> $\delta$ on acceptance          | 0.1           |
| $\gamma_{\text{rej}}$ (gamma-rej) | factor to <i>increase</i> $\delta$ on rejection         | 10.0          |
| $\epsilon$ (epsilon)              | floor for diagonal elements (prevents division by zero) | $10^{-6}$     |



LM convergence (top panel) and optimisation trajectories on Wheeler's Ridge (bottom panel). The diagonal-scaling variant

### Why Diagonal Scaling Helps

Plain damping adds the same  $\delta$  to every diagonal entry, treating all dimensions equally. Diagonal scaling adds  $\delta H_{ii}$  to the  $i$ -th diagonal, so directions with high curvature receive more damping. This is especially helpful when the Hessian is poorly conditioned (eigenvalues span many orders of magnitude), which is common in machine learning and physics-based optimisation.

# Levenberg-Marquardt: Python Implementation

Listing 3: Levenberg-Marquardt algorithm in Python (NumPy)

```
1 import numpy as np
2 from numpy.linalg import solve
3
4 def levenberg_marquardt(f, grad_f, hess_f, x0,
5                         delta=1.0, gamma_acc=0.1, gamma_rej=10.0,
6                         eps=1e-6, max_iter=100):
7     x = np.array(x0, dtype=float)
8     history = [x.copy()]
9     for _ in range(max_iter):
10         g = grad_f(x)
11         if np.linalg.norm(g) < eps:
12             break
13         H = hess_f(x)
14         # Diagonal scaling: take component-wise max with eps
15         d = np.maximum(np.diag(H), eps)
16         M = H + delta * np.diag(d)
17         try:
18             step = solve(M, g)
19         except np.linalg.LinAlgError:
20             step = g # fallback: gradient step
21         x_new = x - step
22         if f(x_new) < f(x):
23             x = x_new
24             delta *= gamma_acc # accept: reduce damping
25     else:
```

## 6.4 LM for Sum of Squares (Nonlinear Least Squares) I

A very common problem in science and engineering is **nonlinear least squares**: fitting a parametric model to data by minimising the sum of squared residuals (the differences between observed and predicted values). Although this is a special case of general optimisation, its structure allows us to approximate the Hessian cheaply using only first-order information – and the approximation is always positive semi-definite, avoiding the indefinite Hessian problem that plagues plain Newton's method.

### Objective: Sum of Squared Residuals

Let  $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$  ( $f$  sub  $i$ ) be the  $i$ -th residual function – for example,  $f_i(\mathbf{x}) = \text{model}_i(\mathbf{x}) - y_i$  where  $y_i$  is the  $i$ -th observation and  $\text{model}_i(\mathbf{x})$  is the model prediction. The objective is:

$$f(\mathbf{x}) = \sum_{i=1}^m f_i(\mathbf{x})^2$$

where the sum runs over all  $m$  ( $m$ ) data points or residuals. We want to find  $\mathbf{x} \in \mathbb{R}^n$  that makes this total squared error as small as possible.

## 6.4 LM for Sum of Squares (Nonlinear Least Squares) II

### Gradient of the Sum-of-Squares Objective

Differentiating with respect to  $\mathbf{x}$  using the chain rule:

$$\nabla f(\mathbf{x}) = 2 \sum_{i=1}^m f_i(\mathbf{x}) \nabla f_i(\mathbf{x})$$

Here  $\nabla f_i(\mathbf{x}) \in \mathbb{R}^n$  (nabla  $f$  sub  $i$ ) is the gradient of the  $i$ -th residual – a vector of partial derivatives of the  $i$ -th residual with respect to each parameter. This is the  $i$ -th row of the **Jacobian matrix  $\mathbf{J}$**  (bold  $\mathbf{J}$ ), which stacks all residual gradients into a single  $m \times n$  matrix.

## 6.4 LM for Sum of Squares (Nonlinear Least Squares) III

### Linearising the Residuals (Gauss-Newton Idea)

If we approximate each residual as linear in  $\mathbf{x}$  around the current iterate  $\mathbf{x}^{(k)}$ :

$$f_i(\mathbf{x}) \approx f_i(\mathbf{x}^{(k)}) + \nabla f_i(\mathbf{x}^{(k)})^\top (\mathbf{x} - \mathbf{x}^{(k)})$$

and substitute into the gradient formula, we obtain an approximation of the gradient and Hessian using only first derivatives:

$$\tilde{\mathbf{g}}^{(k)} = \sum_i f_i(\mathbf{x}^{(k)}) \nabla f_i(\mathbf{x}^{(k)}) = \mathbf{J}^\top \mathbf{r}$$

$$\tilde{\mathbf{H}}^{(k)} = \sum_i \nabla f_i(\mathbf{x}^{(k)}) \nabla f_i(\mathbf{x}^{(k)})^\top = \mathbf{J}^\top \mathbf{J}$$

where  $\mathbf{r} = [f_1(\mathbf{x}^{(k)}), \dots, f_m(\mathbf{x}^{(k)})]^\top$  (bold  $\mathbf{r}$ ) is the residual vector and  $\mathbf{J}$  (bold  $\mathbf{J}$ ) is the Jacobian matrix with entry  $J_{ij} = \partial f_i / \partial x_j$ .

## 6.4 LM for Sum of Squares (Nonlinear Least Squares) IV

### Key Advantage: No True Second Derivatives Needed

The approximate Hessian  $\tilde{\mathbf{H}}^{(k)} = \mathbf{J}^\top \mathbf{J}$  is an *outer product* (a sum of rank-1 matrices), which means it is always **positive semi-definite** (all eigenvalues  $\geq 0$ ). In particular, it never has negative eigenvalues, so the Gauss-Newton step is always at worst a neutral direction – never an uphill direction. This is a huge advantage over full Newton, which can produce saddle-point steps when the Hessian has negative eigenvalues. Furthermore, building  $\mathbf{J}^\top \mathbf{J}$  requires only first derivatives  $\partial f_i / \partial x_j$ , which are typically far cheaper to compute than second derivatives.

## 6.4 LM for Sum of Squares (Nonlinear Least Squares) V

### LM Update for Sum of Squares

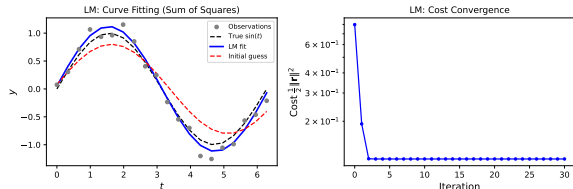
Incorporating the LM diagonal damping into the Gauss-Newton update:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \left( \mathbf{J}^\top \mathbf{J} + \delta \text{diag}(\mathbf{J}^\top \mathbf{J}) \right)^{-1} \mathbf{J}^\top \mathbf{r}$$

When residuals are small at the solution (which happens in well-fitting models),  $\mathbf{J}^\top \mathbf{J}$  is an excellent approximation to the true Hessian, and the LM update gives near-Newton convergence rates while remaining stable throughout.



## 6.4 LM for Sum of Squares (Nonlinear Least Squares) VI



Left: LM fitting the model  $a \sin(bt + c)$  ( $a$  times sine of  $(bt + c)$ ) to noisy observations (blue dots). The fitted curve (solid line) agrees closely with the true signal. Right: the objective value

$f(\mathbf{x}) = \sum_i f_i(\mathbf{x})^2$  decreases rapidly over iterations.

### Comparison: LM vs. Full Newton

| Property                       | LM (sum-of-sq.)                  | Full Newton                                  |
|--------------------------------|----------------------------------|----------------------------------------------|
| Hessian approx.                | $\mathbf{J}^T \mathbf{J}$        | exact $\nabla^2 f$                           |
| Always positive semi-definite? | ✓                                | No                                           |
| Cost per iter.                 | $O(mn^2)$                        | $O(n^3)$                                     |
| Derivatives needed             | $\mathbf{J}$ only<br>(1st order) | $\mathbf{J}$ and $\nabla^2 f$<br>(2nd order) |

$m$  = number of residuals,  $n$  = problem dimension. When  $m \gg n$  (many data points, few parameters),  $O(mn^2)$  is dominated by the Jacobian construction and the overall cost is far lower than full Newton.

# LM for Sum of Squares: Python with SciPy

Listing 4: LM for nonlinear least squares using `scipy.optimize.least_squares`

```
1 import numpy as np
2 from scipy.optimize import least_squares
3
4 # --- Curve fitting: model  $a \sin(b \cdot t + c)$  ---
5 np.random.seed(42)
6 t_obs = np.linspace(0, 2*np.pi, 20)
7 y_obs = np.sin(t_obs) + 0.15 * np.random.randn(len(t_obs))
8
9 def model(params, t):
10     a, b, c = params
11     return a * np.sin(b * t + c)
12
13 def residuals(params):
14     return model(params, t_obs) - y_obs # vector of  $f_i(x)$ 
15
16 # SciPy uses LM (method='lm') or trust-region approaches
17 result = least_squares(residuals, x0=[0.8, 0.9, 0.1], method='lm')
18
19 print(f"Fitted params:    a={result.x[0]:.4f}, b={result.x[1]:.4f}, c={result.x[2]:.4f}")
20 print(f"True params:      a=1.0000, b=1.0000, c=0.0000")
21 print(f"Final residual:    {0.5 * result.cost:.6f}")
22 print(f"Function evals:     {result.nfev}")
23
24 # Manual approach: build Jacobian explicitly
25 def jacobian(params, t):
```

## 6.5 Quasi-Newton Methods: The Big Idea I

Full Newton's method computes the Hessian  $\mathbf{H}$  exactly at every iteration, costing  $O(n^2)$  gradient evaluations and  $O(n^3)$  arithmetic to solve the resulting linear system. For large problems ( $n \geq 10^4$  or so) this is prohibitive. **Quasi-Newton methods** avoid computing the Hessian altogether. Instead, they maintain an approximation  $\mathbf{Q}^{(k)}$  (bold Q at iteration  $k$ ) to the *inverse* Hessian  $(\mathbf{H}^{(k)})^{-1}$ , which they update cheaply at each step using only gradient information. Because the update costs  $O(n^2)$  arithmetic and the approximation improves over time, these methods achieve *superlinear* convergence (faster than linear, slower than quadratic) without ever forming or inverting the full Hessian.

### Quasi-Newton Update Rule

The iterate update is:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{Q}^{(k)} \mathbf{g}^{(k)}$$

where  $\mathbf{Q}^{(k)} \approx (\mathbf{H}^{(k)})^{-1}$  is the current inverse-Hessian approximation. Compare to Newton's update  $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - (\mathbf{H}^{(k)})^{-1} \mathbf{g}^{(k)}$ : the structure is identical; the difference lies entirely in how  $\mathbf{Q}$  is obtained.

## 6.5 Quasi-Newton Methods: The Big Idea II

### The Secant Condition

After each step we have new information: the gradient changed from  $\mathbf{g}^{(k)}$  to  $\mathbf{g}^{(k+1)}$  as we moved from  $\mathbf{x}^{(k)}$  to  $\mathbf{x}^{(k+1)}$ . We require the updated approximation  $\mathbf{Q}^{(k+1)}$  to satisfy the **secant condition**:

$$\mathbf{Q}^{(k+1)} \boldsymbol{\delta}^{(k)} = \boldsymbol{\gamma}^{(k)}$$

where:

- $\boldsymbol{\delta}^{(k)} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$  (delta superscript k, bold) is the *step vector* – how far we just moved.
- $\boldsymbol{\gamma}^{(k)} = \mathbf{g}^{(k+1)} - \mathbf{g}^{(k)}$  (gamma superscript k, bold) is the *gradient difference* – how much the gradient changed.

The secant condition says: the new inverse-Hessian approximation, when applied to the step we just took, should predict the gradient change we just observed. This is the multidimensional analogue of the secant approximation to  $f''$ . With  $n$  dimensions this condition provides  $n$  equations but  $n^2$  unknowns in  $\mathbf{Q}^{(k+1)}$ , so there are infinitely many solutions – different quasi-Newton variants impose additional criteria (symmetry, positive definiteness, minimum change) to pick a unique update.

## 6.5 Quasi-Newton Methods: The Big Idea III

### Summary: Three Main Variants

| Method | Update rank                        | Always PSD?                 | Memory cost |
|--------|------------------------------------|-----------------------------|-------------|
| SR1    | Rank-1 (one outer product)         | No                          | $O(n^2)$    |
| DFP    | Rank-2 (two outer products)        | Yes (under curvature cond.) | $O(n^2)$    |
| BFGS   | Rank-2 (dual of DFP)               | Yes (under curvature cond.) | $O(n^2)$    |
| L-BFGS | Rank-2 (implicit, limited history) | Yes                         | $O(mn)$     |

PSD = positive semi-definite;  $m$  = history length (typically 3–20).

# DFP Update (Davidon-Fletcher-Powell) I

The DFP method (named after William Davidon, Roger Fletcher, and Michael Powell, who developed it in the 1960s) was the first widely used quasi-Newton method. At each iteration it performs a *rank-2 update* of the inverse-Hessian approximation  $\mathbf{Q}$  – meaning the new  $\mathbf{Q}$  differs from the old one by a matrix of rank at most 2 (the sum of two rank-1 outer-product matrices). This keeps the update cheap ( $O(n^2)$ ) while incorporating information from both the step and the gradient change.

# DFP Update (Davidon-Fletcher-Powell) II

## DFP Update Formula

$$\mathbf{Q}^{(k+1)} = \mathbf{Q}^{(k)} - \frac{\mathbf{Q}^{(k)}\boldsymbol{\gamma}\boldsymbol{\gamma}^\top\mathbf{Q}^{(k)}}{\boldsymbol{\gamma}^\top\mathbf{Q}^{(k)}\boldsymbol{\gamma}} + \frac{\boldsymbol{\delta}\boldsymbol{\delta}^\top}{\boldsymbol{\delta}^\top\boldsymbol{\gamma}}$$

### Symbol guide:

- $\mathbf{Q}^{(k)}$  (bold Q): the current inverse-Hessian approximation (an  $n \times n$  positive definite matrix).
- $\boldsymbol{\delta} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$  (bold delta): the step vector.
- $\boldsymbol{\gamma} = \mathbf{g}^{(k+1)} - \mathbf{g}^{(k)}$  (bold gamma): the gradient difference vector.
- $\boldsymbol{\gamma}^\top\mathbf{Q}^{(k)}\boldsymbol{\gamma}$ : a scalar quadratic form – a positive scalar if  $\mathbf{Q}^{(k)}$  is positive definite.
- $\boldsymbol{\delta}^\top\boldsymbol{\gamma}$ : a scalar curvature measure; must be positive (the *curvature condition*) for the update to preserve positive definiteness.
- The first fraction subtracts the component of  $\mathbf{Q}$  that is “inconsistent” with the new gradient information; the second fraction adds a new outer product that satisfies the secant condition exactly.

# DFP Update (Davidon-Fletcher-Powell) III

## DFP Algorithm Steps

- 1 Initialise:  $\mathbf{Q}^{(1)} = \mathbf{I}_n$  (start with the identity matrix as the inverse-Hessian approximation – this is equivalent to a gradient descent step at iteration 1).
- 2 Compute search direction:  $\mathbf{d}^{(k)} = -\mathbf{Q}^{(k)} \mathbf{g}^{(k)}$ .
- 3 Perform a line search: find  $\alpha^* = \arg \min_{\alpha} f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)})$  (alpha-star, the optimal step length).
- 4 Update:  $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^* \mathbf{d}^{(k)}$ .
- 5 Compute  $\delta = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$  and  $\gamma = \mathbf{g}^{(k+1)} - \mathbf{g}^{(k)}$ .
- 6 Apply the DFP formula to obtain  $\mathbf{Q}^{(k+1)}$ .
- 7 Repeat from step 2 until convergence.

## Properties

If  $\delta^T \gamma > 0$  (the curvature condition holds, meaning the gradient increased in the direction we stepped), then:

- $\mathbf{Q}^{(k+1)}$  remains positive definite – guaranteed

## Curvature Condition $\delta^T \gamma > 0$

This condition says that the gradient and the step point in the same “direction” (their inner product is positive). It is guaranteed when the line search satisfies the Wolfe conditions. If it fails, skip the DFP update formula and use the



# BFGS Update (Broyden-Fletcher-Goldfarb-Shanno) I

The BFGS method (named after Charles Broyden, Roger Fletcher, Donald Goldfarb, and David Shanno, who independently discovered it in 1970) is currently **the most widely used quasi-Newton method** in general optimisation software. It can be derived as the “dual” of DFP by swapping the roles of  $\delta$  and  $\gamma$ , which gives it superior numerical stability when line searches are performed inexactly.

# BFGS Update (Broyden-Fletcher-Goldfarb-Shanno) II

## BFGS Update Formula

$$\mathbf{Q}^{(k+1)} = (\mathbf{I} - \rho \boldsymbol{\delta} \boldsymbol{\gamma}^\top) \mathbf{Q}^{(k)} (\mathbf{I} - \rho \boldsymbol{\gamma} \boldsymbol{\delta}^\top) + \rho \boldsymbol{\delta} \boldsymbol{\delta}^\top$$

where  $\rho = \frac{1}{\boldsymbol{\delta}^\top \boldsymbol{\gamma}}$  (rho, the reciprocal of the curvature measure).

### Symbol guide:

- $\rho$  (rho): a positive scalar that normalises the curvature;  $\rho = 1/(\boldsymbol{\delta}^\top \boldsymbol{\gamma})$  where  $\boldsymbol{\delta}^\top \boldsymbol{\gamma}$  is the inner product of step and gradient change.
- $\mathbf{I} - \rho \boldsymbol{\delta} \boldsymbol{\gamma}^\top$ : a rank-1 correction to the identity matrix; it “rotates”  $\mathbf{Q}^{(k)}$  to be more consistent with the observed gradient change.
- $\rho \boldsymbol{\delta} \boldsymbol{\delta}^\top$ : a rank-1 matrix added to ensure the secant condition is satisfied.

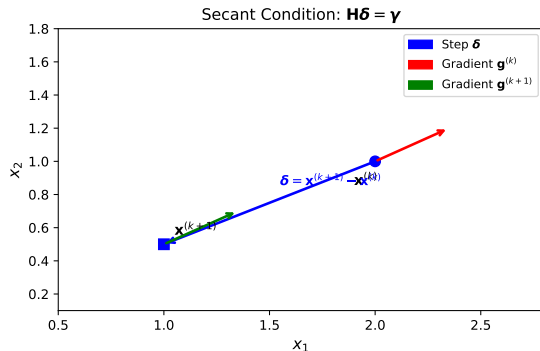
# BFGS Update (Broyden-Fletcher-Goldfarb-Shanno) III

## How to Read the BFGS Formula

Think of it as three steps: (1) the term  $(\mathbf{I} - \rho \delta \gamma^\top)$  on the right “de-rotates” the old approximation; (2)  $\mathbf{Q}^{(k)}$  applies the old inverse Hessian; (3) the left factor and the added term together re-orient the result to satisfy the secant condition. The net effect is a rank-2 update that is often more numerically stable than DFP because it spreads the correction symmetrically around  $\mathbf{Q}$ .

## Why BFGS Outperforms DFP in Practice

- BFGS is the *dual* of DFP (the roles of  $\delta$  and  $\gamma$  are swapped), but this seemingly minor change dramatically improves robustness with inexact line searches.
- With an exact line search both DFP and BFGS produce identical iterates on quadratic functions; with inexact search BFGS almost always wins.
- BFGS has better self-correcting properties: even if one update is inaccurate, later updates



# BFGS: Python Implementation

Listing 5: BFGS quasi-Newton method in Python

```
1 import numpy as np
2 from numpy.linalg import norm
3
4 def bfgs(f, grad_f, x0, eps=1e-8, max_iter=200):
5     """BFGS quasi-Newton minimization (NumPy implementation)."""
6     x = np.array(x0, dtype=float)
7     n = len(x)
8     Q = np.eye(n)          # initial inverse Hessian approx = I
9     history = [f(x)]
10
11     for _ in range(max_iter):
12         g = grad_f(x)
13         if norm(g) < eps:
14             break
15         d = -Q @ g          # descent direction
16
17         # Backtracking line search (Armijo condition)
18         alpha, c1 = 1.0, 1e-4
19         for _ in range(50):
20             if f(x + alpha*d) <= f(x) + c1*alpha*(g @ d):
21                 break
22             alpha *= 0.5
23
24         x_new = x + alpha * d
25         g_new = grad_f(x_new)
```

## SR1 Update: Symmetric Rank-1 I

The SR1 (Symmetric Rank-1) update is the simplest quasi-Newton update: it modifies  $\mathbf{Q}$  by adding only a single outer-product matrix (a rank-1 matrix). Although it is simpler than BFGS, it does *not* guarantee positive definiteness, which can occasionally cause the method to explore non-descent directions. Despite this, SR1 is used in some trust-region frameworks because its rank-1 structure makes it easier to capture the non-convex (saddle-like) geometry that trust-region methods need to navigate.

# SR1 Update: Symmetric Rank-1 II

## SR1 Update Formula

$$\mathbf{Q}^{(k+1)} = \mathbf{Q}^{(k)} + \frac{(\boldsymbol{\delta} - \mathbf{Q}^{(k)}\boldsymbol{\gamma})(\boldsymbol{\delta} - \mathbf{Q}^{(k)}\boldsymbol{\gamma})^\top}{(\boldsymbol{\delta} - \mathbf{Q}^{(k)}\boldsymbol{\gamma})^\top \boldsymbol{\gamma}}$$

### Symbol guide:

- $\boldsymbol{\delta} - \mathbf{Q}^{(k)}\boldsymbol{\gamma}$  (bold delta minus bold Q times bold gamma): the *residual vector* – the difference between the actual step and the step the current approximation would predict. If  $\mathbf{Q}^{(k)}$  were the true inverse Hessian this would be zero.
- The numerator  $(\boldsymbol{\delta} - \mathbf{Q}^{(k)}\boldsymbol{\gamma})(\boldsymbol{\delta} - \mathbf{Q}^{(k)}\boldsymbol{\gamma})^\top$  is a symmetric rank-1 outer product.
- The denominator  $(\boldsymbol{\delta} - \mathbf{Q}^{(k)}\boldsymbol{\gamma})^\top \boldsymbol{\gamma}$  is a scalar; the update is undefined (division by zero) when this is zero.

**Interpretation:** the update adds a rank-1 matrix that “corrects” the current approximation in the direction of the prediction residual. Only one outer product is needed (rank-1), whereas DFP and BFGS use two (rank-2).

# SR1 Update: Symmetric Rank-1 III

## SR1 Properties and Use Cases

- **Simplicity:** requires only one outer product per iteration.
- **No positive-definiteness guarantee:** the updated  $\mathbf{Q}^{(k+1)}$  may have negative eigenvalues.
- **Better non-convex approximation:** because it is not forced to remain positive definite, SR1 can sometimes better approximate Hessians with negative eigenvalues (near saddle points).
- **Trust-region pairing:** often paired with a trust-region step limit to keep the iterate safe even when  $\mathbf{Q}$  is indefinite.

## Skip Condition for SR1

The SR1 update is numerically unstable or undefined when:

$$\left| (\delta - \mathbf{Q}^{(k)}\gamma)^\top \gamma \right| < \epsilon \cdot \|\delta - \mathbf{Q}^{(k)}\gamma\| \cdot \|\gamma\|$$

(the denominator is small relative to the numerator components). In practice: skip the update whenever the denominator is below a threshold  $\epsilon$  (epsilon) times the norms of the constituent vectors.

## SR1 Update: Symmetric Rank-1 IV

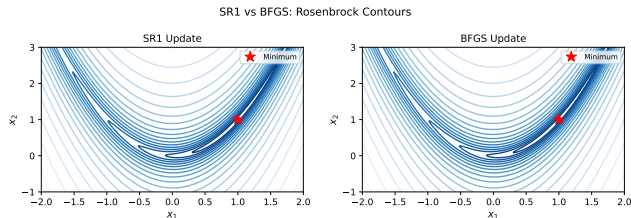
### Comparison of All Quasi-Newton Update Rules

| Method | Rank | Always PD | With inexact LS | Memory   | Convergence |
|--------|------|-----------|-----------------|----------|-------------|
| SR1    | 1    | No        | Moderate        | $O(n^2)$ | superlinear |
| DFP    | 2    | Yes       | Poor            | $O(n^2)$ | superlinear |
| BFGS   | 2    | Yes       | Excellent       | $O(n^2)$ | superlinear |
| L-BFGS | 2    | Yes       | Good            | $O(mn)$  | superlinear |

PD = positive definite; LS = line search;  $m$  = history size (typical: 3–20). All quasi-Newton methods achieve superlinear convergence (faster than linear but slower than quadratic), compared to quadratic convergence for full Newton's method.



# SR1 Update: Symmetric Rank-1 V



Contour plots of the Rosenbrock banana function  $f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$ . Both SR1 and BFGS successfully navigate the curved valley, though BFGS typically takes fewer iterations on this problem.

## 6.5 L-BFGS: Limited-Memory BFGS I

The full BFGS method stores an  $n \times n$  matrix  $\mathbf{Q}$  and updates it at each iteration, costing  $O(n^2)$  memory and  $O(n^2)$  arithmetic per step. For  $n = 10^6$  (one million parameters, as in a medium-sized deep learning model), storing a dense  $n \times n$  matrix would require  $10^{12}$  numbers – completely infeasible.

**L-BFGS** (Limited-memory BFGS) solves this by never forming  $\mathbf{Q}$  explicitly. Instead, it stores only the  $m$  most recent  $(\delta, \gamma)$  pairs ( $m$  is typically 5–20) and uses a clever **two-loop recursion** to compute the matrix-vector product  $\mathbf{Q}\mathbf{g}$  implicitly in  $O(mn)$  time. The rest of the algorithm is identical to BFGS.

## 6.5 L-BFGS: Limited-Memory BFGS II

### L-BFGS: Two-Loop Recursion

Given the  $m$  most recent pairs  $\{(\delta^{(i)}, \gamma^{(i)})\}$  stored in a circular buffer, and the current gradient  $\nabla f(\mathbf{x})$ , the two-loop recursion computes the descent direction  $\mathbf{d} \approx -\mathbf{Q}\nabla f(\mathbf{x})$  as follows:

**Backward pass** (iterates from most recent to oldest,  $i = m$  down to 1): this pass computes scalar weights  $\alpha^{(i)}$  (alpha superscript i) and modifies a running vector  $\mathbf{q}$ :

$$\alpha^{(i)} = \frac{(\delta^{(i)})^\top \mathbf{q}^{(i)}}{(\delta^{(i)})^\top \gamma^{(i)}}, \quad \mathbf{q}^{(i-1)} = \mathbf{q}^{(i)} - \alpha^{(i)} \gamma^{(i)}$$

Starting with  $\mathbf{q}^{(m)} = \nabla f(\mathbf{x})$  (the current gradient).

**Initial scaling**: after the backward pass, scale the result:

$$\mathbf{z}^{(0)} = \frac{(\delta^{(m)})^\top \gamma^{(m)}}{(\gamma^{(m)})^\top \gamma^{(m)}} \mathbf{q}^{(0)}$$

This scaling (the ratio of two inner products) normalises the initial Hessian approximation to roughly the true Hessian scale – a key stabilisation step.

# L-BFGS: Python Implementation

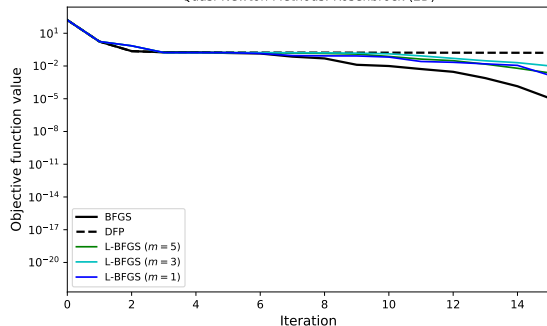
Listing 6: L-BFGS with two-loop recursion in Python

```
1 import numpy as np
2 from collections import deque
3
4 def lbfgs(f, grad_f, x0, m=5, eps=1e-8, max_iter=200):
5     """L-BFGS quasi-Newton minimization."""
6     x = np.array(x0, dtype=float)
7     deltas = deque(maxlen=m)      # step differences s_i
8     gammas = deque(maxlen=m)      # gradient differences y_i
9     history = [f(x)]
10
11     for _ in range(max_iter):
12         g = grad_f(x)
13         if np.linalg.norm(g) < eps:
14             break
15
16         # ---- Two-loop recursion ----
17         q = g.copy()
18         alphas = []
19         for dk, yk in zip(reversed(deltas), reversed(gammas)):
20             rho_k = 1.0 / (dk @ yk + 1e-15)
21             a = rho_k * (dk @ q)
22             q = q - a * yk
23             alphas.append((a, rho_k, dk, yk))
24         alphas.reverse()
```

# Quasi-Newton Methods: Full Comparison I

Having studied SR1, DFP, BFGS, and L-BFGS, we can now compare them side-by-side on a benchmark problem and discuss when to use each. The Rosenbrock banana function  $f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$  is a standard stress test: its global minimum is at  $[1, 1]^T$  but the function has a curved, narrow valley that requires an optimiser to track a nearly parabolic path. Gradient descent struggles here; quasi-Newton methods handle it well.

Quasi-Newton Methods: Rosenbrock (2D)



Objective value vs. iteration for BFGS, DFP, and L-BFGS with

## Key Observations

BFGS and DFP often perform similarly on well-conditioned, smooth problems with exact line search. The advantage of BFGS over DFP becomes clear with inexact (approximate) line search: BFGS's self-correcting property keeps the approximation accurate even when the line search is only approximate. L-BFGS with  $m = 1$  essentially stores no history at all and behaves like a scaled steepest descent. As  $m$  grows from 1 to 5 to 20, L-BFGS approaches BFGS performance asymptotically. For most large-scale problems

## 6.6 Chapter Summary I

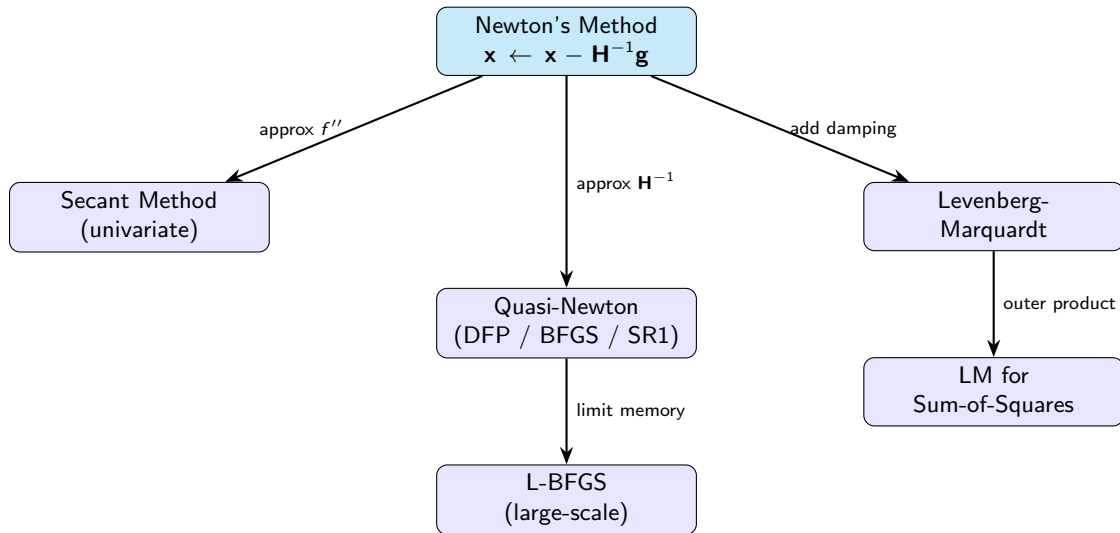
This chapter has presented a family of optimisation algorithms that share a common theme: using curvature information (second-order information) to take more informed steps than gradient descent alone. Each method represents a different trade-off between computational cost, robustness, and convergence speed.

## 6.6 Chapter Summary II

### What We Learned

- **Second-order information** (the Hessian  $\mathbf{H}$ , nabla-squared  $f$ ) enables us to build a quadratic local model of  $f$  and jump directly to its minimum, rather than following the gradient by small steps. This is why second-order methods are much faster near a minimum.
- **Newton's method** updates as  $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - (\mathbf{H}^{(k)})^{-1} \mathbf{g}^{(k)}$  (minus H-inverse times gradient), achieves *quadratic* convergence near a local minimum (error squares at each step), and converges in a single step on exactly quadratic functions. However, it requires computing and inverting the  $n \times n$  Hessian at every iteration ( $O(n^3)$  cost), and it fails when  $\mathbf{H}$  is indefinite.
- **The secant method** approximates  $f''$  from two successive gradient evaluations, avoiding the need for second derivatives. It achieves superlinear convergence of order  $\approx 1.618$  and is applicable to univariate problems.
- **Levenberg-Marquardt** adds damping  $\delta \mathbf{I}$  (or  $\delta \text{diag}(\mathbf{H})$ ) to the Hessian before inverting, interpolating between gradient descent (large  $\delta$ ) and Newton's method (small  $\delta$ ). Adaptive  $\delta$  adjustments make it robust globally while still converging fast near the minimum.
- **LM for sum of squares** replaces the true Hessian with the Gauss-Newton approximation  $\mathbf{J}^\top \mathbf{J}$  – always positive semi-definite and requiring only first-order Jacobian information. This is the workhorse of nonlinear least-squares fitting in practice.

# Method Family Tree





## 6.7 Selected Exercises I

### Exercise 6.1: Second-Order vs. First-Order Information

**Question.** What advantage does second-order information provide about convergence that first-order information lacks?

**Answer.** Second-order information (the Hessian  $\mathbf{H}$ ) can *certify* that a stationary point is a local minimum: a point where  $\nabla f(\mathbf{x}) = \mathbf{0}$  (gradient is zero) is a local minimum if and only if  $\mathbf{H}$  is positive definite there. First-order information alone cannot distinguish between a local minimum, a local maximum, and a saddle point – all three have zero gradient. Moreover, the Hessian directly provides the curvature information needed to determine an optimal step size, enabling quadratic convergence.

## 6.7 Selected Exercises II

### Exercise 6.9: One Newton Step on a Quadratic

**Question.** Starting from the origin, apply one step of Newton's method to  $f(\mathbf{x}) = (x_1 + 1)^2 + (x_2 + 3)^2 + 4$ .

**Answer.** At  $\mathbf{x}^{(1)} = [0, 0]^\top$ :

$$\nabla f(\mathbf{x}^{(1)}) = \begin{bmatrix} 2(0 + 1) \\ 2(0 + 3) \end{bmatrix} = \begin{bmatrix} 2 \\ 6 \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = 2\mathbf{I}.$$

The Newton step is:

$$\mathbf{x}^{(2)} = [0, 0]^\top - (2\mathbf{I})^{-1}[2, 6]^\top = [0, 0]^\top - [1, 3]^\top = [-1, -3]^\top.$$

We have found the global minimum  $\mathbf{x}^* = [-1, -3]^\top$  in a single step. This is expected because  $f$  is exactly quadratic, so the quadratic approximation is perfect everywhere. The minimum value is  $f(\mathbf{x}^*) = 0 + 0 + 4 = 4$ .

## 6.7 Selected Exercises III

### Exercise 6.7: Advantage of Quasi-Newton over Full Newton

**Question.** What is the advantage of a quasi-Newton method over Newton's method?

**Answer.** Quasi-Newton methods avoid computing the Hessian matrix  $\mathbf{H}$  and solving the  $n \times n$  linear system  $\mathbf{H} \mathbf{d} = -\mathbf{g}$  at every iteration. This is important for three reasons:

- 1 **Cost per iteration:** solving  $\mathbf{H} \mathbf{d} = -\mathbf{g}$  requires  $O(n^3)$  arithmetic (via LU or Cholesky decomposition); the BFGS update costs only  $O(n^2)$ .
- 2 **Derivative availability:** many practical problems do not have analytical second derivatives; automatic differentiation of second derivatives can be expensive.
- 3 **Memory:** for  $n = 10^6$ , the Hessian has  $10^{12}$  entries and cannot be stored.

The trade-off is that quasi-Newton methods achieve superlinear convergence rather than the quadratic convergence of full Newton – still much faster than gradient descent.

## 6.7 Selected Exercises IV

### Exercise 6.2: Newton vs. Bisection for Root Finding

**Question.** When finding roots in one dimension, when would we use Newton's method instead of the bisection method?

**Answer.** Newton's method is preferred when:

- The derivative  $f'$  is available and cheap to compute.
- High accuracy is needed quickly: Newton converges quadratically (the number of correct decimal digits roughly doubles each step), while bisection converges only linearly (adding about one bit of precision per step).
- A good starting point close to the root is available, so the quadratic approximation is accurate.

Bisection is preferred when:

- Derivatives are unavailable.
- Guaranteed convergence is essential: bisection always converges if the function is continuous and the root is bracketed.
- Low accuracy is sufficient (bisection is simpler to implement reliably).

### Exercise 6.8: When BFGS Update Does Not Exist

# Quick Reference: Second-Order Methods

| Method              | Update Rule                                                                                                                                                         | Requirements           | Convergence |                               |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------|-------------------------------|
| Newton's Method     | $\mathbf{x} \leftarrow \mathbf{x} - \mathbf{H}^{-1} \mathbf{g}$                                                                                                     | $\nabla f, \nabla^2 f$ | Quadratic   |                               |
| Secant Method       | $x \leftarrow x - \frac{x - x_{\text{prev}}}{g - g_{\text{prev}}} g$                                                                                                | $f'$ only              | Superlinear |                               |
| Levenberg-Marquardt | $\mathbf{x} \leftarrow \mathbf{x} - (\mathbf{H} + \delta D)^{-1} \mathbf{g}$                                                                                        | $\nabla f, \nabla^2 f$ | Adaptive    |                               |
| LM Sum of Squares   | $\mathbf{x} \leftarrow \mathbf{x} - (\mathbf{J}^\top \mathbf{J} + \delta D)^{-1} \mathbf{J}^\top \mathbf{r}$                                                        | $\mathbf{J}$ only      | Adaptive    | $D = \text{diag}(\mathbf{H})$ |
| DFP                 | $\mathbf{Q} \leftarrow \mathbf{Q} - \frac{\mathbf{Q} \gamma \gamma^\top \mathbf{Q}}{\gamma^\top \mathbf{Q} \gamma} + \frac{\delta \delta^\top}{\delta^\top \gamma}$ | $\nabla f$ only        | Superlinear |                               |
| BFGS                | $\mathbf{Q} \leftarrow (\mathbf{I} - \rho \delta \gamma^\top) \mathbf{Q} (\mathbf{I} - \rho \gamma \delta^\top) + \rho \delta \delta^\top$                          | $\nabla f$ only        | Superlinear |                               |
| L-BFGS              | Two-loop recursion with $m$ history pairs                                                                                                                           | $\nabla f$ only        | Superlinear |                               |

or  $\mathbf{I}$ ;  $\rho$  (rho) =  $1/(\delta^\top \gamma)$ ;  $\mathbf{r}$  = residual vector;  $\mathbf{J}$  = Jacobian matrix.